

IA PARA DEVS

DESENVOLVIMENTO DE ML NA CLOUD

AULA 03

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

EMSE

O QUE VEM POR AÍ?

Vimos, até agora, diversos desafios envolvidos ao treinar modelos de aprendizagem de máquina, como problemas de limitação de infraestrutura. Vimos, também, algumas formas de como a computação em nuvem pode nos ajudar a vencer esses desafios. Uma coisa que não vimos, no entanto, foi como escalar recursos disponíveis na nuvem para acelerar nosso desenvolvimento e testes de modelos de aprendizagem de máquina. Quanto mais rápido pudermos iterar entre esses modelos, mais rápido chegaremos a um resultado satisfatório para podermos aplicar esses modelos em nossos problemas reais.

Dado este contexto, nesta aula abordaremos diversas formas onde podemos utilizar recursos de nuvem para acelerar nossos desenvolvimentos de modelos de aprendizado de máquina.

HANDS ON

Nesta aula vamos ver algumas formas onde podemos utilizar os recursos de nuvem para acelerar nossos desenvolvimentos de aprendizado de máquina. Além de utilizar recursos mais robustos (como máquinas com mais memória por exemplo), podemos dar mais velocidade aos treinamentos dos modelos utilizando recursos em paralelo. Apresentaremos algumas formas de utilizar recursos de forma paralela para o treinamento de modelos.

A primeira, é treinar diversos modelos diferentes ao mesmo tempo utilizando recursos distintos (como máquinas virtuais). Gerenciar diversos treinamentos ao mesmo tempo pode ser uma tarefa complexa, com isso, apresentaremos como gerenciar esses treinamentos por meio do AutoML. A próxima forma é treinar o mesmo modelo pré-definido, porém com diferentes hiperparâmetros, a fim de identificar quais hiperparâmetros são os mais eficazes para a nossa tarefa. Neste caso, utilizaremos as ferramentas de hyperparameter tuning disponíveis no Azure para nos ajudar gerenciar os processos. Por fim, mostraremos como podemos utilizar recursos paralelos para treinar um único modelo com hiperparâmetros já definidos.

Para acompanhar este hands on, acesse o [repositório](#) com os materiais.

SAIBA MAIS

Vimos, até aqui, algumas formas de utilizar recursos paralelos para acelerar o desenvolvimento de modelos de aprendizado de máquina. No entanto, existem alguns detalhes que permitem que esse tipo de capacidade funcione. Nesta seção, apresentaremos alguns conteúdos relacionados a características e técnicas que permitem o processamento paralelo de cargas de aprendizado de máquina.

CPU vs GPU vs TPU

É possível utilizar diferentes tipos de hardwares para realizar o treinamento de modelos de aprendizado de máquina, como CPUs, GPUs e TPUs. No entanto, existem diferenças e capacidades para cada um deles. A seguir, listamos as características de cada um desses tipos de hardware.

CPU (Unidade de Processamento Central)

A CPU é o cérebro do computador e é responsável por executar instruções e processar dados. Ela contém núcleos de processamento que são altamente capazes de executar uma ampla variedade de tarefas. As CPUs modernas têm múltiplos núcleos (2, 4, 6, 8 ou mais) para realizar várias tarefas simultaneamente, usando técnicas de paralelismo.

A CPU é otimizada para tarefas sequenciais, executando uma série de instruções de maneira rápida e eficiente. Os núcleos de uma CPU são versáteis e podem lidar com uma variedade de operações, desde cálculos complexos até operações simples de entrada e saída.

Vantagens:

- Versatilidade: pode executar uma ampla gama de tarefas, desde operações básicas do sistema até tarefas computacionais intensivas.
- Flexibilidade: ideal para tarefas que não são altamente paralelizáveis.

Desvantagens:

- Menor eficiência para cargas de trabalho intensivas em dados, como aprendizado de máquina, devido à limitação de paralelismo.

GPU (Unidade de Processamento Gráfico)

A GPU é projetada especificamente para lidar com processamento gráfico e tarefas altamente paralelizáveis, como renderização de imagens e vídeos.

A GPU contém milhares de núcleos menores que podem lidar com cálculos simultâneos. É altamente eficiente em realizar operações matriciais, o que é crucial para muitos algoritmos de aprendizado de máquina, como redes neurais.

Vantagens:

- Grande poder de processamento paralelo, o que é útil para tarefas intensivas em dados como treinamento de modelos de aprendizado de máquina.
- Eficiente para operações matriciais comuns em algoritmos de Machine Learning.

Desvantagens:

- Menos flexibilidade em comparação com CPUs, sendo mais específica para cargas de trabalho altamente paralelizáveis.

TPU (Unidade de Processamento Tensorial)

O TPU (Tensor Processing Unit) é uma unidade especializada desenvolvida pela Google para acelerar operações tensoriais, essenciais para o processamento de dados em aprendizado de máquina e IA.

O TPU é otimizado para manipular operações em tensores, estruturas de dados cruciais em modelos de aprendizado de máquina, especialmente redes neurais. A arquitetura do TPU é projetada para maximizar a eficiência na execução de operações matriciais.

Vantagens:

- Extremamente eficiente para operações tensoriais, acelerando o treinamento e a execução de modelos de aprendizado de máquina.
- Especialização em operações de tensores oferece um desempenho superior em comparação com CPUs e GPUs genéricas para essas tarefas específicas.

Desvantagens:

- Menos versátil do que CPUs e GPUs, sendo altamente especializado em tarefas relacionadas ao aprendizado de máquina.

Comparação para cargas de aprendizado de máquina

- **CPU:** é adequada para uma variedade de tarefas, mas pode ser menos eficiente para treinamentos massivos de modelos de aprendizado de máquina devido à limitação de paralelismo.
- **GPU:** oferece um aumento significativo no desempenho para tarefas altamente paralelizáveis, como treinamento de redes neurais, mas pode ser menos flexível para outras tarefas.
- **TPU:** especializado em operações tensoriais, o TPU supera tanto CPUs quanto GPUs em tarefas relacionadas ao aprendizado de máquina, oferecendo um desempenho superior.

Enquanto a CPU oferece versatilidade, a GPU é eficiente em paralelismo e a TPU se destaca em tarefas específicas de aprendizado de máquina envolvendo operações tensoriais.

Etapas de um AutoML

Na primeira parte da nossa videoaula, utilizamos o recurso de AutoML para nos ajudar treinar e gerenciar diferentes modelos a fim de encontrar o modelo e configuração mais eficaz para o nosso problema. No entanto, não aprofundamos sobre como um processo de AutoML funciona. Aqui listamos algumas etapas importantes que um AutoML deve ser capaz de realizar.

Seleção de Algoritmos

Nesta etapa, um sistema (potencialmente um modelo de aprendizado de máquina) identifica automaticamente os melhores algoritmos de aprendizado de máquina para um determinado conjunto de dado. Para isso, este sistema leva em consideração características dos dados, tamanho do conjunto de dados e o tipo de problema, como classificação ou regressão.

Engenharia de Características

Os dados utilizados para o treinamento de um modelo de aprendizado de máquina podem possuir centenas ou milhares de características. Utilizar todas essas características junto a um modelo pode não ser uma boa idéia, pois vai exigir um processamento muito maior em relação a custo e tempo para que os padrões possam ser extraídos destes dados, além de correr um risco maior de ficar preso em um mínimo local devido por cauda de um ajuste, impossibilitando a criação de um modelo eficiente. Um processo tradicional de treinamento de um modelo de aprendizado de máquina está ligado a selecionar as características dos dados mais promissoras para realizar o treinamento. Ao utilizar uma ferramenta de AutoML, é possível delegar essa etapa, logo o sistema de AutoML precisa ter uma capacidade de automatizar a seleção de características relevantes dos dados para melhorar o desempenho do modelo. Isso pode envolver a criação de novas características, seleção das mais importantes ou a transformação de característica existentes.

Busca de Arquitetura de Modelos

Além de selecionar os algoritmos mais promissores, também é necessário adequar suas configurações de operação (como hiperparâmetros), e no caso de modelos de aprendizado profundo, a sua arquitetura. Com isso, é necessário que um sistema de AutoML explore automaticamente a arquitetura dos modelos como redes neurais para encontrar a melhor estrutura. Para isso, é possível fazer uso de técnicas como busca aleatória, busca em grade, otimização baseada em gradiente e aprendizado por reforço. Este é um problema complexo e frequentemente alvo de pesquisas acadêmicas.

Empilhamento e Comitê de Métodos

Uma outra capacidade frequentemente utilizada por sistemas de AutoML é a agregação de diferentes modelos em um único modelo. Isso é realizado por meio da combinação automática de vários modelos para melhorar o desempenho geral. Alguns métodos como bagging, boosting e modelos de ensemble (comitês) são usados para criar uma previsão final com base em múltiplos modelos. Logo podemos criar diferentes modelos, analisar seus desempenhos e depois criar novos modelos facilmente realizando diferentes tipos de combinação com os modelos já treinados.

Técnicas de Hyperparameter Tuning

Para descobrir a melhor configuração para um tipo de modelo pré-determinado, fizemos o uso de ferramentas de regulagem de hiperparâmetros (hyperparameter tuning). Embora possamos ter uma intuição de que tipos de modelos e configurações funcionarão bem para o nosso problema, na prática são necessárias algumas rodadas de experimentos para confirmação (ou pelo menos para se aproximar das configurações ideais). Como realizar esses testes pode ser uma tarefa demorada e custosa, mas existem diferentes técnicas para nos ajudar selecionar os hiperparâmetros para serem testados:

Busca em grade

A busca em grade utiliza uma lista de valores para cada hiperparâmetro a ser ajustado, e com isso testa todas as combinações possíveis de valores de hiperparâmetros.

- **Vantagens:** simplicidade conceitual, garante a busca completa no espaço de hiperparâmetros especificado.
- **Desvantagens:** pode ser computacionalmente caro, especialmente com muitos hiperparâmetros ou valores.

Busca Aleatória

A busca aleatória seleciona aleatoriamente um conjunto de valores para os hiperparâmetros em cada iteração. Neste caso, são realizadas várias tentativas com diferentes combinações de hiperparâmetros.

- **Vantagens:** menos custoso computacionalmente do que a busca em grade, especialmente em espaços de hiperparâmetros grandes.
- **Desvantagens:** não garante a exploração completa do espaço de busca.

Otimização Bayesiana

A otimização Bayesiana utiliza modelos probabilísticos para estimar quais configurações de hiperparâmetros são mais promissoras. Neste método, fazemos iterativamente ajustes em um modelo probabilístico com base nos resultados anteriores para guiar a busca.

- **Vantagens:** eficiente na exploração de espaços de hiperparâmetros, tende a convergir mais rapidamente.
- **Desvantagens:** pode ser mais complexo de implementar, exige uma boa modelagem probabilística. Além disso, pode dificultar o processamento paralelo, uma vez que novos testes dependem de resultados anteriores.

Otimização baseada em gradiente

A otimização baseada em gradiente usa técnicas de otimização baseadas em gradientes para encontrar os melhores hiperparâmetros. Neste método, os hiperparâmetros são otimizados usando informações de gradientes ou derivadas.

- **Vantagens:** eficiente em espaços de hiperparâmetros contínuos e diferenciáveis.
- **Desvantagens:** não é adequado para hiperparâmetros discretos ou não diferenciáveis.

Algoritmos Evolutivos

O método de algoritmos evolutivos emprega técnicas inspiradas na evolução biológica para otimizar os hiperparâmetros. São utilizadas técnicas como seleção natural, crossover e mutação para evoluir conjuntos de hiperparâmetros.

- **Vantagens:** pode encontrar boas soluções em espaços de busca complexos ou não lineares.
- **Desvantagens:** pode exigir mais iterações para convergir e ser computacionalmente caro em espaços de alta dimensionalidade.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, apresentamos diferentes formas de acelerar o processo de desenvolvimento de modelos de aprendizado de máquina, por meio do uso de recursos computacionais paralelos. Primeiramente, contextualizamos o problema e, em seguida, discutimos diferentes formas de paralelizar o processamento de cargas de aprendizado de máquina. Discutimos como treinar diferentes modelos ao mesmo tempo utilizando ferramentas de AutoML, em seguida apresentamos como treinar um mesmo modelo com diferentes hiperparâmetros por meio de ferramentas de Hyperparameter Tuning. Por fim, apresentamos diferentes formas de paralelizar o treinamento de um único modelo, a fim de acelerar sua convergência.

REFERÊNCIAS

AWS. **Qual a diferença entre GPUs e CPUs?**. 2023. Disponível em: <<https://aws.amazon.com/pt/compare/the-difference-between-gpus-cpus>>. Acesso em: 21 mar. 2024.

Google Cloud Platform. **Tensor Processing Units**. 2023. Disponível em: <<https://cloud.google.com/tpu/docs/tpus>>. Acesso em: 21 mar. 2024.

Microsoft Azure. **Fazer ajuste de hiperparâmetro em um modelo**. 2023. Disponível em: <<https://learn.microsoft.com/pt-br/azure/machine-learning/how-to-tune-hyperparameters>>. Acesso em: 21 mar. 2024.

Microsoft Azure. **O que é AutoML?** 2023. Disponível em: <<https://learn.microsoft.com/pt-br/azure/machine-learning/concept-automated-ml>>. Acesso em: 21 mar. 2024.

PALAVRAS-CHAVE

Automl. Hiperparameter Tuning. Aprendizado de Máquina Distribuido.

EMAP



POSTECH